Section 4 — Agent Memory

How Agents Remember

Author: Suresh D R | AI Product Developer & Technology Mentor

---

Table of Contents

#	Topic
4.1	Why Memory Matters
4.2	Short-Term Memory — In-Context
4.3	Long-Term Memory — External Storage
4.4	Episodic Memory — Past Experiences
4.5	Semantic Memory — Facts and Knowledge
4.6	Memory Operations — How Memory Actually Works
4.7	Memory Summarisation — When Context Gets Too Long
4.8	Cross-Session Persistence — Memory That Survives
4.9	State Management — Saving and Restoring Progress
4.10	Section Summary

What This Section Is About

Section 2 taught how agents act. Section 3 taught how agents think. Section 4 teaches how agents **remember**.

Memory is the difference between an agent that feels like a brilliant but forgetful assistant and one that feels like a trusted colleague who knows your work, your preferences, your history, and learns from every interaction.

Without memory, every conversation starts from absolute zero. The agent does not know who you are. It does not remember what you discussed yesterday. It cannot learn from mistakes it made last week. Every time you talk to it, it is meeting you for the first time.

With memory, the agent builds up knowledge over time. It recalls past conversations. It avoids mistakes it already made. It knows your preferences without you repeating them. It feels genuinely personal.

By the end of this section you will understand four types of memory, how each one works, how agents store and retrieve information, and how memory persists across sessions.

4.1 Why Memory Matters

> **Analogy:** Imagine hiring a personal assistant who has amnesia. Every single morning they wake up with no memory of working with you before. You have to re-introduce yourself. You have to re-explain your preferences. You have to re-tell them everything you discussed yesterday. They make the same mistakes as last week because they have no record of what went wrong. This is an agent without memory — brilliant in the moment, useless over time.

The Problem Without Memory

Here is what happens without memory in a real conversation:

```
Day 1, Session 1:
  You: "My name is Arjun. I am managing a retail chain with 12 stores across Mumbai."
  Agent: "Great! How can I help you Arjun?"
  You: "Help me plan delivery routes for next week across Pune and Mumbai."
  Agent: [generates optimised delivery routes]

Day 2, Session 2 (new session — no memory):
  You: "Continue where we left off."
  Agent: "I don't have any context about previous conversations.
          Could you tell me what you are working on?"

  You have to start over. Every. Single. Time.
```

And it gets worse:

```
Week 1: Agent makes an error — gives wrong fuel cost calculation
        You correct it.

Week 2: Same question asked again.
        Agent gives the same wrong calculation.
        It has no memory of the correction.
        You correct it again.

Week 3: Same mistake. Same correction.
        The agent never learns.
```

This is the fundamental limitation of agents without memory. They are stateless — every session is a clean slate with no knowledge of what came before.

What Memory Enables

With memory properly implemented, here is what changes:

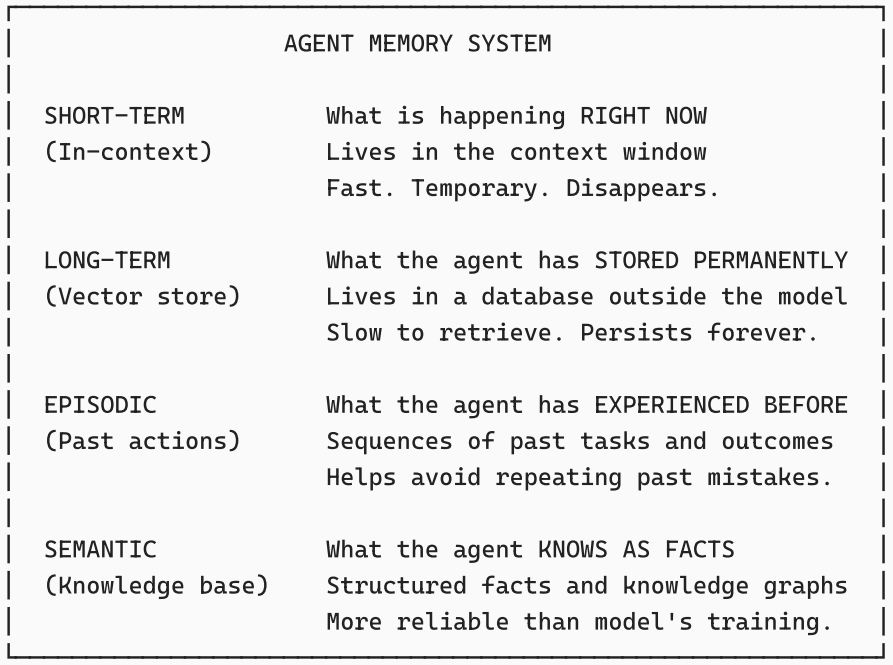
```
You: "Continue where we left off on the delivery planning."
Agent: "Of course. Last session we completed the route planning for Pune.
        You mentioned the Thane routes need special attention for traffic.
        Shall I start with the Mumbai routes?"

You: "Yes. And remember I prefer Indian examples."
Agent: "Noted. Here is Section 4 with Indian examples..."

Next session:
You: "Write the next section."
Agent: "Based on our previous sessions, we have planned routes for
        Pune and Thane. Still pending: Mumbai Central and Bandra zones.
        You prefer avoiding toll roads where possible. Shall I continue?"
```

The agent knows you. It knows your work. It knows your preferences. It builds on past sessions instead of starting fresh. That is the power of memory.

The Four Types of Agent Memory



Each type serves a different purpose. A well-designed agent uses all four together.

4.2 Short-Term Memory — In-Context

> **Analogy:** Your working desk. Everything currently in front of you — papers, notes, your laptop, your coffee. You can see it all immediately. But your desk has limited space. When it fills up, you have to move older things off to make room. And when you leave the office at the end of the day, everything on the desk is cleared. Short-term memory works exactly like your desk.

What Is Short-Term Memory?

Short-term memory is everything the agent can see right now in the current session. It includes:

- The original goal or question
- Every message exchanged in this conversation
- Every tool call made and every result received
- Any instructions given during this session

All of this lives inside the **context window** — the maximum amount of text the LLM can see at one time.

```
What the agent holds in short-term memory right now:

  [System message]   "You are a helpful AI assistant..."
  [User message 1]   "My name is Arjun"
  [Agent reply 1]    "Nice to meet you Arjun!"
  [User message 2]   "I run a logistics company"
  [Agent reply 2]    "That sounds interesting..."
  [Tool call]        web_search("Mumbai delivery route optimisation 2024")
  [Tool result]      "Results: ..."
  [User message 3]   "What did I tell you my name was?"
  [Agent reply 3]    "You told me your name is Arjun."  ← remembers!
```

The agent answered the last question correctly because everything is still in the context window.

Context Window Limits — The Problem

Context windows have a maximum size measured in **tokens** (roughly words):

Model	Context Window	Approx Words
GPT-3.5-turbo	16,000 tokens	12,000 words
GPT-4o	128,000 tokens	96,000 words
Claude 3.5 Sonnet	200,000 tokens	150,000 words
Gemini 1.5 Pro	1,000,000 tokens	750,000 words

Sounds like a lot. But for a long-running agent doing many tasks over many sessions, even 128,000 tokens fills up quickly.

```
What fills the context window:
  System prompt              ~500 tokens
  Conversation history       grows with every message
  Tool call inputs           each call adds tokens
  Tool results               search results can be 1000+ tokens each
  Agent's reasoning          each Thought step adds tokens

After 20 tool calls with search results:
  Each result: ~800 tokens
  20 results:  16,000 tokens → already at GPT-3.5 limit
  Agent starts forgetting earlier messages.
```

When the context window fills up, the oldest content is dropped. The agent starts forgetting what happened at the beginning of the conversation.

How Short-Term Memory Is Managed

Strategy 1 — Conversation Buffer (keep everything)
Keep the full conversation history in the context. Simple. Works for short conversations. Fails for long ones.

```
Session start:  [message 1] [message 2]
After 5 turns:  [msg 1] [msg 2] [msg 3] [msg 4] [msg 5]
After 20 turns: [msg 1] ... [msg 20] — context nearly full
After 30 turns: oldest messages start getting dropped
```

Strategy 2 — Sliding Window (keep recent)
Keep only the last N messages. Older messages are dropped. Agent always has the most recent context but forgets the distant past.

```
Window size: 10 messages

Turn 15: [msg 6] [msg 7] [msg 8] [msg 9] [msg 10] [msg 11]
         [msg 12] [msg 13] [msg 14] [msg 15]
         (messages 1–5 dropped)

Fast. Simple. Agent loses early context but remains responsive.
```

Strategy 3 — Summary Buffer (summarise old, keep recent)
When the conversation gets long, summarise the old part and keep the recent part in full. Agent has a compressed version of the past plus full detail of recent messages.

```
First 20 messages → summarised to 3 sentences
Last 5 messages   → kept in full

Agent sees:
  [Summary: "User is Arjun, logistics company, 200 agents in Maharashtra.
             Completed Pune and Thane routes. Prefers toll-free roads."]
  [Recent message 16 in full]
  [Recent message 17 in full]
  [Recent message 18 in full]
  [Recent message 19 in full]
  [Recent message 20 in full]

Context is compact but agent retains all important context.
This is the best strategy for most production agents.
```

4.3 Long-Term Memory — External Storage

> **Analogy:** Your filing cabinet at the office. It does not sit on your desk. It is in the corner. You cannot see everything inside it at once. But when you need something — a document from six months ago, a note from a past meeting — you open it and search. You find what you need and bring it to your desk. Long-term memory works exactly like that filing cabinet.

What Is Long-Term Memory?

Long-term memory is information stored permanently outside the model in a database. It persists across sessions — it is there tomorrow, next week, next year. The agent retrieves relevant memories when needed by searching the database.

```
Short-term memory:  lives inside the model (context window)
                    fast, immediately visible
                    disappears when session ends
```

```
Long-term memory:   lives outside the model (database)
                    requires a search to retrieve
                    persists permanently
```

How Long-Term Memory Is Stored — Vector Databases

Long-term memory is stored in a **vector database**. This is a special type of database designed for storing and searching text based on meaning (not just keywords).

Why a Vector Database?

Regular databases search by exact match — find rows where name = "Arjun". This does not work for memories because you rarely know the exact words. You want to find memories that are **semantically similar** to what you are looking for.

```
You ask: "What are my preferences for writing style?"

Regular database search: look for rows containing "preferences" AND "writing style"
Result: might miss relevant rows that say "user likes formal tone" or
        "user wants Indian examples" — different words, same meaning

Vector database search: convert "preferences writing style" into a vector,
                        find all memories whose vectors are close to it
Result: finds "user likes formal tone", "user prefers Indian examples",
        "user asked to avoid complex jargon" — all relevant, different words
```

What Is a Vector?

A vector is a list of numbers that represents the meaning of a piece of text.

```
"The dog ran fast"    → [0.23, -0.87, 0.44, 0.12, ...]
"The puppy ran quickly" → [0.25, -0.84, 0.43, 0.14, ...]
"The stock market rose" → [0.89, 0.23, -0.67, 0.91, ...]

The first two sentences have similar meanings → their vectors are close
```

```
The third sentence has different meaning → its vector is far away

Distance between vectors = difference in meaning
Close vectors = similar meaning
Far vectors = different meaning
```

When the agent wants to retrieve a memory, it converts the current query into a vector and finds the stored memories whose vectors are closest.

How Long-Term Memory Works — Step by Step

```
STORING A MEMORY:

  1. Something important happens in the conversation
     Example: "User's name is Arjun. He runs a logistics company with 200 delivery agents in Maharashtra."

  2. This text is converted into a vector using an embedding model
     "Arjun... logistics... 200 agents... Maharashtra" → [0.23, -0.45, 0.88, ...]

  3. The vector + original text is stored in the vector database
     Database now has this memory saved permanently

_____________________________________________________________

RETRIEVING A MEMORY (next session, or later):

  1. Agent receives a question: "Who am I building this course for?"

  2. This question is converted into a vector
     "Who... building... course... for" → [0.21, -0.44, 0.87, ...]

  3. Database searched for vectors closest to this query vector

  4. Closest match found: "User's name is Arjun. Runs logistics company with 200 delivery agents."

  5. This memory is pulled into the context window
```

```
6. Agent now has the memory and can answer the question
```

Common Vector Databases

Database	Best For	Free?
FAISS	Local, fast, no server needed	Yes
Chroma	Simple local setup, good for learning	Yes
Pinecone	Cloud, production scale	Free tier
Weaviate	Cloud, advanced features	Free tier
Qdrant	Cloud, high performance	Free tier

For learning and local development — use **Chroma** or **FAISS**.
For production — use **Pinecone** or **Weaviate**.

4.4 Episodic Memory — Past Experiences

> **Analogy:** A cricket batsman's match diary. After every innings they write down what deliveries troubled them, which bowlers they struggled against, what worked and what did not. Before the next match they read these notes. They go to the crease already knowing what mistakes to avoid. Episodic memory gives agents the same advantage.

What Is Episodic Memory?

Episodic memory stores **sequences of past actions and their outcomes**. Not just facts, but the full experience — what the agent tried, what happened, what worked, what failed, and what was learned.

The key difference from other memory types:

```
Semantic memory stores:  FACTS
  "Python was created by Guido van Rossum in 1991"

Episodic memory stores:  EXPERIENCES
  "On 15 March 2024, I was asked to write a Python sorting function.
   I wrote a quicksort implementation. Tests failed because I did not
   handle duplicate values. I fixed it by adding a check.
   Lesson: always test with duplicates when writing sorting functions."
```

Facts tell the agent what is true. Experiences tell the agent what to do and what to avoid.

How Episodic Memory Works

```
CREATING AN EPISODE:

  When: after any significant task attempt
  What gets stored:
    - The task description
    - The approach taken
    - The outcome (success or failure)
    - The lesson learned
    - The timestamp

  Example episode stored:
    {
      "task":     "Write a function to count word frequency",
      "approach": "Used split() and a dictionary",
      "outcome":  "Failed - empty string caused KeyError",
      "lesson":   "Always check for empty input before processing",
      "date":     "2024-03-15"
    }

_____________________________________________________________

USING AN EPISODE (next time similar task appears):
```

```
  Agent receives: "Write a function to find the most common word"

  Agent searches episodic memory:
    Query: "word frequency function python"
    Found episode: "count word frequency → failed on empty string"

  Agent reads the lesson: "Always check for empty input"

  Agent writes the new function with empty input check already included
  from the very first attempt — no failure needed this time
```

Real Example — Episodic Memory Preventing a Repeated Mistake

```
EPISODE STORED ON DAY 1:
  Task: Research top AI companies and write a brief
  Approach: Searched for each company individually
  Problem: Ran out of steps before finishing all 5 companies
  Lesson: When researching multiple companies, search for all of them
          in one query first, then read details one by one.
          Do not search individually — it uses too many steps.

_____________________________________________________________


DAY 2 — same type of task arrives:
  "Research the top 5 electric vehicle companies in India and write a brief"

  WITHOUT episodic memory:
    Agent searches each company separately → runs out of steps → incomplete brief

  WITH episodic memory:
    Agent searches memory → finds the Day 1 episode
    Agent reads the lesson: "search all companies in one query first"
    Agent starts with: web_search("top 5 EV companies India 2024")
    Gets all 5 names at once → then researches each → completes brief within step limit
```

The mistake from Day 1 never happens on Day 2. The agent learned from experience.

4.5 Semantic Memory — Facts and Knowledge

> **Analogy:** An encyclopaedia. It does not store experiences or ongoing conversations. It stores structured, verified facts — definitions, dates, relationships, data. When you need a fact you look it up. Semantic memory is the agent's encyclopaedia.

What Is Semantic Memory?

Semantic memory stores structured **facts and knowledge** — things that are true about the world, about your business, about your domain. Unlike episodic memory (which stores experiences), semantic memory stores stable facts.

```
Episodic memory (experience):
  "Last time I used the email tool, it failed because
   the recipient field was empty. Lesson: always validate
   the email address before calling send_email."

Semantic memory (fact):
  "Company X's CEO is Rahul Sharma."
  "The Python sort() function sorts in ascending order by default."
  "GST in India for software services is 18%."
  "The deadline for project Y is March 31, 2025."
```

Why Semantic Memory Is More Reliable Than Model Training

The LLM's internal knowledge comes from training data with a cutoff date. It can be outdated and sometimes just wrong (hallucination).

Semantic memory solves both problems:

```
WITHOUT semantic memory (relying on LLM training):
  Question: "What is the current CEO of Company X?"
  LLM: "The CEO is [name from training data]"
  Problem: That person resigned 6 months ago. Training data is outdated.
```

```
WITH semantic memory (verified facts stored externally):
  Agent searches semantic memory
  Finds: "CEO of Company X is Rahul Sharma (updated March 2024)"
  Agent answers: "Based on our records, CEO is Rahul Sharma"
  This fact was verified and stored — much more reliable.
```

Knowledge Graphs — Structured Semantic Memory

A knowledge graph stores facts as **entities and relationships**:

```
Entity: Arjun Sharma
  Relationship: [is a] Logistics Company Owner
  Relationship: [manages] 200 delivery agents
  Relationship: [operates in] Maharashtra
  Relationship: [uses] Route Planning Agent

Entity: Logistics Company
  Relationship: [has] 12 delivery zones
  Relationship: [serves] Mumbai, Pune, Thane
  Relationship: [employs] 200 agents
  Relationship: [handles] 5000 deliveries per day

Entity: Mumbai Central Zone
  Relationship: [is part of] Logistics Company
  Relationship: [has] 45 delivery agents
  Relationship: [peak hours] 9 AM to 11 AM, 5 PM to 8 PM
```

The agent can traverse this graph — "Find everything related to the AI Training Course" — and get a complete picture of how entities relate to each other.

4.6 Memory Operations — How Memory Actually Works

> **Analogy:** Think of memory like a librarian. The librarian receives new books (encoding), decides which ones to keep permanently on the shelves (consolidation), finds books when you ask for them (retrieval), updates a book's information when it becomes outdated (updating), and removes books that are

> no longer relevant (forgetting). Every one of these operations happens in an agent's memory system.

The Five Memory Operations

Operation 1 — Encoding

What: Converting new information into a format that can be stored.

For short-term memory, encoding is automatic — anything said in the conversation is immediately in context.

For long-term memory, encoding involves creating an embedding vector from the text and storing it in the database.

```
New information: "The project deadline is March 31, 2025"

Encoding:
  1. Convert to embedding: [0.23, -0.45, 0.88, 0.12, ...]
  2. Store in database with original text and timestamp
  3. Now searchable and retrievable in future sessions
```

Operation 2 — Consolidation

What: Deciding what is worth keeping long-term vs what can be forgotten.

Not everything that happens in a conversation is worth storing permanently. The agent (or the system around it) decides what is important enough to move to long-term storage.

```
Worth consolidating to long-term memory:
  User's name and role
  User's preferences and communication style
  Important decisions made
  Mistakes and lessons learned
  Key facts about ongoing projects

NOT worth consolidating:
  Small talk ("how are you today?")
  Intermediate calculation steps
```

```
  Tool results that are no longer relevant
  Clarifying questions that were answered
```

Operation 3 — Retrieval

What: Finding relevant memories when needed.

For short-term memory — retrieval is instant, everything is already in context.

For long-term memory — retrieval requires a search. The query is converted to a vector and the database finds the closest matches.

```
Query: "What does this user prefer for formatting?"

Search process:
  1. Convert query to vector
  2. Find top 3 memories closest to this vector
  3. Return: "User prefers bullet points over paragraphs"
             "User wants Indian examples in explanations"
             "User asked for simple English, no jargon"

These 3 memories are now added to the current context.
Agent uses them when writing the response.
```

Operation 4 — Updating

What: Modifying existing memories when information changes.

Facts change. People change roles. Projects change deadlines. If the agent stores outdated information and never updates it, the memory becomes a liability — wrong information confidently stated.

```
Old memory: "The project deadline is March 31, 2025"

User says: "The deadline has moved to April 15, 2025"

Memory system:
  1. Find existing memory about project deadline
  2. Update it: "The project deadline is April 15, 2025 (updated)"
  3. Mark old version as superseded
```

```
Now the agent uses the correct deadline in all future responses.
```

Operation 5 — Forgetting

What: Removing memories that are stale, irrelevant, or incorrect.

Keeping everything forever creates problems — the agent has to search through irrelevant old memories, outdated facts confuse it, and storage grows without limit. Strategic forgetting keeps memory clean and useful.

```
Reasons to forget (remove) a memory:
  Time-based: "Current gold price is Rs 6500" → outdated after a week
  Relevance: Task completed, its details no longer needed
  Superseded: A newer, more accurate memory replaced this one
  Incorrect: Memory was based on wrong information
  User request: "Forget what I told you about that project"
```

4.7 Memory Summarisation — When Context Gets Too Long

> **Analogy:** At the end of a long meeting, someone writes the minutes — a 1-page summary of a 2-hour discussion. Nobody reads the full transcript next time. They read the minutes. The important information is preserved in a fraction of the space. Memory summarisation does the same for agent conversations.

The Problem — Conversations Get Too Long

In a long working session, conversations can run into thousands of messages. The full history cannot fit in the context window. Something has to give.

Three options:

1. Drop old messages entirely → agent loses context
2. Keep everything → context window overflows
3. Summarise old messages → agent keeps the important parts in compressed form

Option 3 is always the right answer.

How Summarisation Works

```
BEFORE SUMMARISATION (conversation is getting long):

  Message 1:  "I run a logistics company with 200 agents"
  Message 2:  "We operate across Mumbai, Pune, and Thane"
  Message 3:  "Avoid toll roads wherever possible"
  Message 4:  "Morning deliveries get priority"
  Message 5:  "Reassign agents if a zone falls behind"
  Message 6:  "Send me a summary at end of each day"
  ... 20 more messages ...
  Message 25: "Now plan routes for the Bandra zone"
  Message 26: "Keep distance under 20 km per agent"

Total: too long for context window

AFTER SUMMARISATION:

  [Summary]: "User is Arjun, logistics company owner with 200 agents
              across Mumbai, Pune, Thane. Preferences:
              avoid tolls, morning deliveries first, daily summary.
              Pune and Thane routes complete. Planning Bandra zone."

  [Recent messages kept in full]:
  Message 24: "Now plan routes for the Bandra zone"
  Message 25: "Keep distance under 20 km per agent"
  Message 26: "Prioritise morning slots for Bandra West"

Context is now compact. Important context is preserved.
Agent continues working as if nothing was lost.
```

Hierarchical Summarisation — For Very Long Conversations

When a conversation runs for many sessions over weeks or months, even summaries can pile up. Hierarchical summarisation creates summaries of summaries.

```
Level 1 — Individual session summaries:
  Session 1 summary: "Planned Pune routes — 35 agents, 420 deliveries"
  Session 2 summary: "Planned Thane routes — 28 agents, 350 deliveries"
  Session 3 summary: "Planned Navi Mumbai — 22 agents, 280 deliveries"
  Session 4 summary: "Planning Bandra zone — in progress"

Level 2 — Weekly summary (summary of session summaries):
  "Week 1: Completed Pune, Thane, Navi Mumbai routes.
   User prefers toll-free roads and morning priority.
   Bandra zone planning in progress."

Level 3 — Project summary (summary of weekly summaries):
  "Logistics Route Project: 3 of 12 zones complete.
   Preferences: toll-free, morning priority, daily summary.
   Next: Bandra and Mumbai Central zones."

Agent always has the high-level context regardless of how
long the project has been running.
```

4.8 Cross-Session Persistence — Memory That Survives

> **Analogy:** A doctor's patient file. Every time you visit, the doctor reads your file — your history, your allergies, your past prescriptions, your ongoing conditions. They do not ask you to re-explain everything from scratch. They pick up where they left off. Cross-session persistence gives agents the same ability.

What Is Cross-Session Persistence?

Short-term memory (context window) disappears when the session ends. Cross-session persistence is memory that **survives** the end of a session and is available when the next session begins.

```
WITHOUT cross-session persistence:
  Session 1: "I run a logistics company with 200 delivery agents"
  Session 2: "Who am I?" → Agent: "I don't know who you are"
  Agent forgot everything. Starts from zero.
```

```
WITH cross-session persistence:
  Session 1: "I run a logistics company with 200 delivery agents"
             → stored in long-term memory
  Session 2: "Who am I?" → Agent searches memory
             → finds "Arjun, runs logistics company with 200 delivery agents"
             → Agent: "You are Arjun, running a logistics company with 200 delivery agents"
  Agent remembers across sessions.
```

User-Specific Memory

In a multi-user system, each user needs their own separate memory space. One user's memories must not bleed into another user's context.

```
User A — Arjun:
  Memory: "Logistics company owner, prefers toll-free routes, morning priority"

User B — Priya:
  Memory: "Running a retail business, needs inventory management help"

When Arjun starts a new session:
  Agent loads ONLY Arjun's memories
  Priya's memories are completely separate and inaccessible

When Priya starts a new session:
  Agent loads ONLY Priya's memories
  Arjun's memories are not visible
```

This is essential for privacy and for giving each user a personalised experience.

Privacy Considerations

Cross-session memory raises important questions about what should and should not be stored.

```
Safe to store long-term:
  User's name and role
  Work preferences (tone, format, language)
  Project context (ongoing work, progress)
  User-approved preferences

Be careful about storing:
  Personal details shared in passing
  Sensitive business information
  Information the user might not want retained

Never store without consent:
  Health information
  Financial details
  Private communications
  Anything the user explicitly said not to store

Always give users the ability to:
  See what is stored about them
  Delete specific memories
  Clear all their data completely
```

4.9 State Management — Saving and Restoring Progress

> **Analogy:** A video game save point. You play for two hours, reach a checkpoint, and the game saves. If your power goes out and you restart, you resume from the last save point. You do not start from the beginning. State management gives agents the same ability — save progress, resume if interrupted.

What Is State Management?

State is everything an agent needs to know to continue a task — what has been done, what tools have been called, what results were received, what is left to do.

Without state management, if an agent is running a 20-step task and crashes at step 12, it has to start over from step 1. All the work from steps 1-11 is lost.

With state management, the agent saves its state after every step. If it crashes at step 12, it restarts and resumes from step 12 — not step 1.

What Gets Saved in State

```
A complete agent state snapshot contains:

  Current task:      "Write a competitor analysis for Zepto"
  Step number:       7 (out of planned 12 steps)
  Steps completed:   [
                       {step: 1, action: "searched Zepto competitors",
                        result: "Blinkit, Swiggy Instamart, BigBasket"},
                       {step: 2, action: "searched Blinkit details",
                        result: "owned by Zomato, 45 min delivery..."},
                       ...up to step 7...
                     ]
  Current plan:      [remaining steps 8-12]
  Memory retrieved:  [relevant memories used so far]
  Status:            "in_progress"
```

This full snapshot is saved to a database (SQLite, Redis, or PostgreSQL) after every step.

State Storage Options

Storage	Speed	Complexity	Best For
JSON file	Fast	None	Learning, simple agents
SQLite	Fast	Low	Single-user agents
Redis	Very fast	Medium	Multi-user, real-time agents
PostgreSQL	Medium	Higher	Production, complex queries

For learning and simple agents: JSON file or SQLite — simple, no setup.

For production agents: Redis (for speed) or PostgreSQL (for complex state).

Checkpointing — Save Points for Long Tasks

Checkpointing is systematic state saving at regular intervals during a long task.

```
Long task: "Research and write a 20-page report on Indian fintech"

WITHOUT checkpointing:
  Agent runs 3 hours → crashes at step 45 of 60
  All 3 hours of work lost
  Agent must start over from step 1

WITH checkpointing (saves every 5 steps):
  Agent saves state after step 5
  Agent saves state after step 10
  Agent saves state after step 15
  ...
  Agent crashes at step 47
  Agent restores from step 45 checkpoint
  Only 2 steps of work lost
  Resumes from step 45 — not step 1
```

Checkpointing is essential for any agent running tasks longer than a few minutes. In production, you never run a long agent task without it.

4.10 Section Summary

Memory Type	What It Stores	Where	Persists?	Best For
Short-term	Current conversation, tool results	Context window (in model)	No — gone when session ends	Active working session
Long-term	Permanent facts, preferences, history	Vector database (outside model)	Yes — forever	Cross-session knowledge
Episodic	Past task experiences, lessons	Database	Yes	Learning from past attempts
Semantic	Verified facts and knowledge	Knowledge base / graph	Yes	Reliable factual information

---

The Key Takeaways

Short-term memory is the agent's working desk — fast, immediately visible, but limited in space and temporary. When the session ends, the desk is cleared.

Long-term memory is the filing cabinet — stores information permanently, requires a search to retrieve, but available across all sessions forever.

Episodic memory is the experience diary — records what the agent tried, what happened, and what was learned. Prevents repeating past mistakes.

Semantic memory is the encyclopaedia — verified, structured facts more reliable than the model's training data. Gets updated when facts change.

Memory operations — encoding, consolidation, retrieval, updating, and forgetting — are the five actions that make a memory system actually work rather than just accumulate noise.

Summarisation solves the context overflow problem — compress old conversation into a summary, keep recent messages in full. The agent keeps important context without hitting the limit.

Cross-session persistence makes the agent feel personal — it knows who you are, what you are working on, and what you prefer, every time you come back.

State management makes the agent robust — save progress after every step, resume from the last checkpoint if something goes wrong. Essential for any long-running task.

How All Four Memory Types Work Together

```
A complete agent memory system in action:

User starts a new session:
  1. Agent loads SHORT-TERM: the current question
  2. Agent retrieves LONG-TERM: past context about this user
  3. Agent checks EPISODIC: any past attempts at this type of task
  4. Agent queries SEMANTIC: any relevant verified facts

Agent runs the task:
  5. SHORT-TERM grows with each step
  6. STATE saved to database after each step (checkpointing)

Task gets long:
  7. SUMMARISATION compresses old context
```

```
  8. Short-term window stays within limits

Task completed:
  9. Important learnings saved to EPISODIC memory
  10. New facts saved to SEMANTIC memory
  11. Key context saved to LONG-TERM memory

Next session:
  Agent starts with all of this already loaded.
  Picks up exactly where it left off.
```

When to Use Each Memory Type — Simple Guide

Memory Type	Use It When	Do NOT Use It When
Short-term	Single session tasks, active conversation	Task spans multiple days or sessions
Long-term	You need the agent to remember across sessions	One-off task with no follow-up
Episodic	Agent repeats similar tasks often	Agent does each task only once
Semantic	You have verified facts the agent must get right	Facts change too frequently to maintain
Summarisation	Conversations run longer than 30 messages	Short focused sessions
State management	Task takes more than 5 minutes to complete	Quick single-step tasks

---

Real Use Cases — Where Each Memory Type Is Used in Production

Use Case 1 — Personal AI Assistant

Someone uses an AI assistant every day for work.

```
Short-term memory:
  Holds the current conversation — today's meeting notes,
  current task, the question just asked.
```

```
Long-term memory:
  Remembers the user's name, role, preferences.
  "Arjun prefers brief summaries. Needs toll-free route options.
   Operates across Mumbai, Pune, Thane."
  Loaded automatically every time Arjun starts a new session.

Episodic memory:
  "Last time Arjun asked for a route plan, he wanted agent counts
   and distances in a table. The paragraph format was rejected."
  Agent uses table format from the very first attempt next time.

Semantic memory:
  Arjun's 12 delivery zones, agent assignments, daily targets.
  Agent knows the logistics context without being told every day.
```

Use Case 2 — Customer Support Agent

An agent handles thousands of customer queries every day.

```
Short-term memory:
  Holds the current customer's question and conversation.
  Each customer gets a fresh context — no bleed between customers.

Long-term memory:
  When a returning customer contacts support, the agent retrieves
  their past orders, past complaints, and past resolutions.
  "This customer had a delivery issue last month. It was resolved
   with a 10% discount." Agent knows this before the conversation starts.

Episodic memory:
  "When customers complain about delayed delivery and are offered
   just an apology, 60% escalate. When offered a discount, only 15% escalate."
  Agent has learned from past interactions what resolution works best.

Semantic memory:
```

```
  Company policies, product details, refund rules, shipping timelines.
  Agent gets these facts right every time — not from training data
  which could be outdated, but from the verified knowledge base.
```

Use Case 3 — Research Agent

An agent helps researchers read papers, summarise findings, and track literature.

```
Short-term memory:
  Currently reading a paper. Holds the paper content and
  the researcher's questions about it.

Long-term memory:
  Every paper summary stored permanently.
  "We already reviewed Smith et al. (2023) on transformer attention.
   Key finding: sparse attention is 3x faster on long sequences."
  Agent never reads the same paper twice.

Episodic memory:
  "Last time I searched for RAG papers I got too many
   irrelevant results. Using 'retrieval augmented generation
   benchmark 2024' as the query works much better."
  Agent uses better search queries from the start next time.

Semantic memory:
  Key concepts, definitions, important papers, authors.
  "RAG = Retrieval Augmented Generation, introduced by Lewis et al. 2020"
  Agent explains concepts consistently and correctly every time.

Summarisation:
  After 20 papers, the conversation history is huge.
  Agent compresses: "Reviewed 20 papers on RAG. Main findings:
  hybrid retrieval outperforms dense-only in 15/20 papers..."
  Session continues without hitting context limits.
```

Use Case 4 — Coding Agent

An agent helps a developer write, review, and debug code over many sessions.

```
Short-term memory:
  Current code file being worked on. Recent error messages.
  Current conversation about the bug.

Long-term memory:
  The full codebase architecture understood from past sessions.
  "This project uses FastAPI + PostgreSQL. Auth is handled in auth.py.
   The developer prefers type hints and docstrings in all functions."

Episodic memory:
  "Last time I suggested using a list comprehension here, the developer
   rejected it — they prefer explicit for loops for readability."
  Agent suggests explicit loops from the start next time.

State management:
  Debugging a complex issue that takes 2 hours.
  Agent saves progress after each debugging step.
  If session times out, developer can resume — agent picks up
  from the last checkpoint instead of starting the debug from scratch.
```

Use Case 5 — Sales and CRM Agent

An agent manages relationships with hundreds of customers.

```
Long-term memory:
  Complete history of every interaction with every customer.
  "Priya from Company ABC called 3 times this month. Interested in
   the enterprise plan. Concerned about data privacy compliance."

Episodic memory:
  "When we send a follow-up email within 24 hours of a demo,
   conversion rate is 40%. When we wait more than 48 hours, it drops to 12%."
```

```
  Agent schedules follow-ups within 24 hours automatically.

Semantic memory:
  Product pricing, features, competitors, compliance certifications.
  Agent answers every sales question accurately without hallucinating.

Cross-session persistence:
  Sales rep hands off a customer to a colleague.
  New agent loads full customer history instantly.
  Colleague continues the conversation seamlessly — no awkward
  "can you remind me what we discussed?" moment.
```

The Simple Decision — Which Memory Do I Need?

```
"Does my agent need to remember things across sessions?"
  No  → short-term memory is enough
  Yes → add long-term memory (vector database)

"Does my agent do the same type of task repeatedly?"
  No  → no episodic memory needed
  Yes → add episodic memory (learn from past attempts)

"Does my agent need to get specific facts exactly right?"
  No  → model's training knowledge is fine
  Yes → add semantic memory (verified facts database)

"Does my agent run tasks longer than 5 minutes?"
  No  → no state management needed
  Yes → add checkpointing (save after every step)

"Do my conversations run longer than 30 messages?"
  No  → no summarisation needed
  Yes → add memory summarisation

Most production agents need ALL of these.
```

```
Start with short-term only.
Add each type as you discover the need for it.
```

Suresh D R — AI Product Developer & Technology Mentor
Agentic AI — Section 4 | Generative AI Training Course